Kodin Code Session Report - Sujal Bharti

Date: 10/29/2026, 2:57:33 PM

QUESTION

A banking transaction monitoring system maintains a shared variable count to record the total number of transactions processed by two worker threads. Each worker thread must increment the shared counter 100,000 times.

Develop a Java program using the Thread class to demonstrate the effects of concurrent access to a shared variable.

Requirements:
1. Create a class Race that extends the Thread class and declare a static integer variable count = 0.
2. Create two threads, t1 and t2, where each thread increments count 100,000 times inside its run() method.
3. Part A Without Synchronization: Implement the program without using synchronized. Start both threads, wait for their completion using join(), and display the expected and actual counter values.
4. Part B With Synchronization: Modify the same program by using an appropriate synchronization mechanism to ensure that only one thread increments the shared counter at a time. Execute the program again and display the expected and actual values.
5. Run both versions multiple times and compare the results.

AIM

To demonstarte the concept of multithreading in Java and observe the effects of thread interference on shared data with and without synchronization

ALGORITHM

Part A: Without Synchronization
1. Create a class Race that extends Thread and declare a static integer count=0.
2. Override run() to increment count 100000 times using a standard for loop (count++).
3. In Main, instantiate two thread objects(t1 and t2).
4. Start both threads using t1.start() and t2.start() to excecute concurrently
5. Wait for termination using t1.join() and t2.join().
6. Print count, observing data loss due to overlapping execution(race condition).

Part B: With synchronization
1. Create a class RaceSynchronized that extends thread and declare a static integer count=0.
2. Override run() to loop 100,000 times placing the increment step inside a synchronized(RaceSynchronized.class) block
3. In main, instantiate two thread objects (t1 and t2)
4. Start both threads using t1.start() and t2.start()
5. Wait for termination using t1.join() and t2.join()
6. Print count, which will accurately display exactly 200,000 due to mutual exclusion

SOLUTIONS

File: s1.java

```
class Race extends Thread{
    public static int count = 0;
```

```
        private boolean useSynchronization;

        public Race(boolean useSynchronization){
            this.useSynchronization = useSynchronization;
        }
        @Override
        public void run(){
            for (int i=0;i<100000; i++){
                count++;
                }
            }
        }
    public class s1{
        public static void main(String[] args){
            Race t1 = new Race(true);
            Race t2 = new Race(false);

            t1.start();
            t2.start();

            try{
                t1.join();
                t2.join();
            }
            catch (InterruptedException e){
                System.out.println("Thread execution interrupted: "+ e.getMessage());
            }
            System.out.println("--- Part A: Without Synchronization ---");
            System.out.println("Expected count value: 200,000");
            System.out.println("Actual count value: " +
String.format("%d",Race.count));
        }
    }
```

File: s2.java

```
    class RaceSynchronized extends Thread{
        public static int count = 0;
        @Override
        public void run(){
            for(int i=0;i<100000;i++){
                synchronized(RaceSynchronized.class){
                    count++;
                }
            }
        }
    }
    public class s2{
        public static void main(String[] args){
            RaceSynchronized t1 = new RaceSynchronized();
            RaceSynchronized t2 = new RaceSynchronized();

            t1.start();
            t2.start();

            try{
                t1.join();
                t2.join();
            }
            catch (InterruptedException e){
                System.out.println("Thread execution interrupted:"+ e.getMessage());
            }
            System.out.println("--- Part B: With Synchronization ---");
            System.out.println("Expected count value: 200,000");
            System.out.println("Actual count value: " +
String.format("%,d",RaceSynchronized.count));
```

```
        }
    }
```

PROGRAM OUTPUT

```
    ========== s1.java ==========
    --- Part A: Without Synchronization ---
    Expected count value: 200,000
    Actual count value: 121360

    ========== s2.java ==========
    --- Part B: With Synchronization ---
    Expected count value: 200,000
    Actual count value: 200,000
```

CONCLUSION

Without Synchronnization we have learn multiple threads accessing shared data simultaneously cause a race condition leading to data corruption and inaccurate results.

With Synchronization we have learn enforcing the mutual exclusion ensures thread safety prevents data loss and guarantes an accurate final outputs

External Activity / Applications Found During Session

1:56:16 PM	msedge	1:57:53 PM	Cam Proctoring: Camera access not allowed